Das Repo als wissenschaftliche Publikation

Für wen?

Studierende, Korpusforschende und Modellgruppen sollen selbstständig von einem Download zu einer nachvollziehbaren Untersuchung gelangen.

Welcher Bestand?

data/tei enthält die maßgeblichen Buchdateien. Lösungen sind mit Rätselkennungen verbunden. Automatische Annotationen liegen getrennt und nennen Verfahren und Prüfstatus.

Wie einsteigen?

README erklärt Bestand, Lizenz, Struktur und ersten Aufruf. Kleine Beispiele, Teilpakete und vollständige Daten sind ohne Uni-Konto lokal nutzbar.

Wie freigeben?

Prüfungen, Bestandsvergleich und gezielte fachliche Lektüre gehen dem Release voraus. Tag, Releasenotizen und Archivpaket bezeichnen die Veröffentlichung.

Wie verbessern?

Fehler werden als Issues gemeldet, Korrekturen geprüft und in neuen Releases beschrieben. Ältere Forschungsstände bleiben referenzierbar; ihre Mängel werden nicht zur Pflicht künftiger Fassungen.

UNSERE EMPFEHLUNG FÜR BUCHDATEIEN

Ein Name, der Menschen und Skripten hilft

```
rk_1786_b0001.xml
```

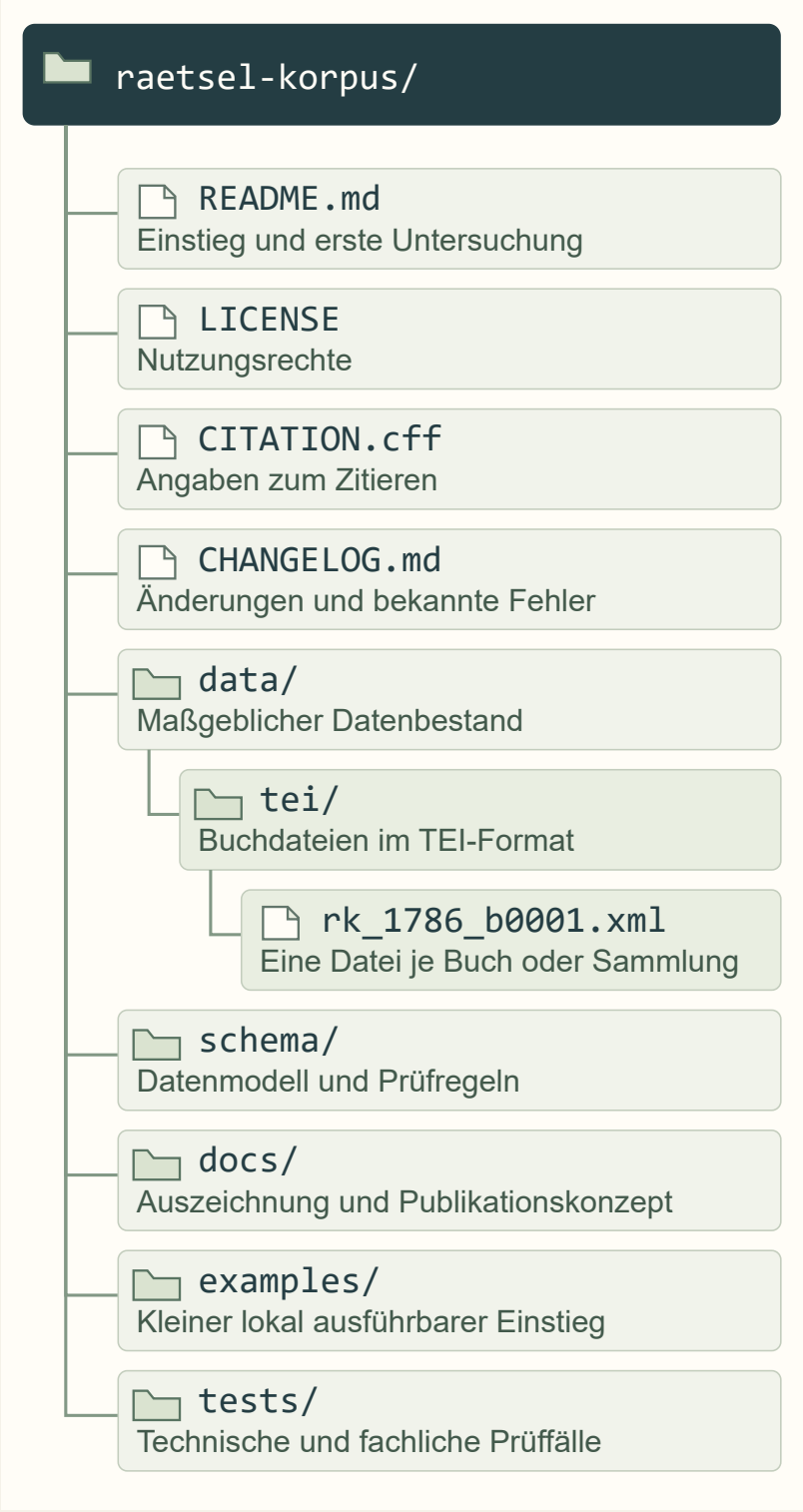
rk	Korpuskürzel
1786	Dokumentiertes Erscheinungsjahr; ohne bekannte Jahresangabe: `undatiert`
b0001	Stabile Buchkennung, auch wenn Titel oder Datierung korrigiert werden

1. **Einheitlich und aussagekräftig:** dieselbe Reihenfolge, feste Stellenzahl und die Endung `.xml`. Titel und ausführliche Quellenbeschreibung stehen in den Metadaten.
2. **Plattformverträglich:** kurze ASCII-Namen, Kleinbuchstaben und relative Pfade. Keine Leerzeichen, Umlaute, Doppelpunkte oder reservierten Gerätenamen; keine Namen, die sich nur in Groß-/Kleinschreibung unterscheiden. Die etablierten Metadateien `README.md`, `LICENSE`, `CITATION.cff` und `CHANGELOG.md` behalten ihre Standardschreibweise.
3. **Für Automatisierungen:** das feste Muster ermöglicht Filter nach Jahr oder Buchkennung. Die Aufnahmeprüfung gleicht diese Angaben mit den maßgeblichen Metadaten ab.
4. **Änderungen geregelt:** Namen nicht beiläufig wechseln. Bei korrigierter Datierung gezielt in Git umbenennen, die stabile Buchkennung erhalten und die Änderung im Changelog erklären.

Filterbeispiel: `rk_1786_*.xml` wählt die entsprechend benannten Buchdateien des Jahres 1786. Unbekannte Datierungen heißen etwa `rk_undatiert_b0004.xml`.

Namensregeln als Markdown ↓ · Lokalen Namensprüfer herunterladen ↓

Unser Publikationsrepo



Grafik als Text lesen

Ordnerstruktur des Korpusrepos

1. raetsel-korpus/
2. README.md
3. Einstieg und erste Untersuchung

4. LICENSE
5. Nutzungsrechte
6. CITATION.cff
7. Angaben zum Zitieren
8. CHANGELOG.md
9. Änderungen und bekannte Fehler
10. data/
11. Maßgeblicher Datenbestand
12. tei/
13. Buchdateien im TEI-Format
14. rk_1786_b0001.xml
15. Eine Datei je Buch oder Sammlung
16. schema/
17. Datenmodell und Prüfregeln
18. docs/
19. Auszeichnung und Publikationskonzept
20. examples/
21. Kleiner lokal ausführbarer Einstieg
22. tests/
23. Technische und fachliche Prüffälle

Planung und Pflege auf einen Blick

Das vollständige Handbuch mit Zielgruppen, Mindestinhalt und Beispielausschnitten steht als repo-handbuch.md bereit.

Eintrag	Anlegen und planen	Pflegen
README.md	Mit dem Publikationskonzept. Die erste Aufgabe einer neuen Nutzerin festlegen. Bei Einrichtung des Publikationsrepos, vor dem ersten Datenrelease.	Bei jeder Änderung an Einstieg, Ordnern, Aufrufen, Lizenz, Zitierweise oder Kontakt im selben Pull Request mitändern. Vor jedem Release den Einstieg von einer anderen Person ausführen lassen.
LICENSE	Bei der Quellenauswahl: Nutzungsbedingungen erfassen und klären, welche Lizenz für welche Projektbestandteile gilt. Vor der ersten öffentlichen Freigabe nach abgeschlossener Rechteklärung.	Bei neuen Quellen oder anders lizenzierten Bestandteilen den Geltungsbereich prüfen. Änderungen fachlich freigeben und in den Releasenotizen erklären; bestehende Lizenztexte nicht beiläufig austauschen.
CITATION.cff	Mit dem Publikationskonzept: Titel, Verantwortliche und Versionsbezug festlegen. Vor dem ersten Release anlegen und mit einem CFF-Prüfwerkzeug validieren.	Bei jedem Release Version und Datum aktualisieren; den bestätigten Versionsidentifikator nach der Archivierung eintragen. Autorenschaften bei relevanten Beiträgen prüfen. Alle Metadaten im Releaseablauf abgleichen.

Eintrag	Anlegen und planen	Pflegen
CHANGELOG.md	Zusammen mit Versionsschema und Releaseweg. Mit der ersten öffentlichen Version. Laufende Änderungen zunächst unter „Unveröffentlicht“ sammeln.	Jeder Pull Request mit nutzbarer Daten-/Modelländerung ergänzt einen Eintrag. Vor dem Release Einträge ordnen, Datum und Version setzen; bekannte Fehler betroffener älterer Fassungen kennzeichnen.
data/	Vor der Aufnahme der ersten Buchdateien: Einheit, Kennungen, Metadaten und Paketaufteilung festlegen. Mit dem ersten geprüften Teilbestand.	Neue oder korrigierte Daten über Pull Requests prüfen und fachlich freigeben. Erst ein Release macht den geprüften Gesamtstand zur bezeichneten Forschungsgrundlage.
schema/	Mit dem TEI-Modell und vor einer großen automatisierten Aufnahme. Sobald das erste Datenmodell geprüft werden kann.	Jede Modelländerung ändert Schema, betroffene Daten, Tests und Dokumentation im selben Änderungsvorschlag. Schemafassung im Release festhalten.
docs/	In der Konzeptphase Gliederung und Verantwortung für Regeln festlegen. Mit dem ersten Modell und den ersten Arbeitsregeln; nicht erst am Projektende nachschreiben.	Bei geänderten Regeln zusammen mit Daten, Schema oder Werkzeugen anpassen. Fachliche Entscheidungen aus Issues in die dauerhafte Dokumentation übernehmen. Vor Releases Widersprüche prüfen.
examples/	Mit dem ersten Nutzungsszenario: Welche Aufgabe soll jemand in wenigen Schritten schaffen? Vor dem ersten öffentlichen Release mit einem kleinen passenden Datenausschnitt.	Wenn Datenmodell, Aufruf oder Abhängigkeiten wechseln, Beispiel und Anleitung im selben Pull Request anpassen. Vor jedem Release frisch ausführen; gelegentlich von einer neuen Person ausprobieren lassen.
tests/	Zusammen mit Konvertierung und Modell: Was darf ein Verarbeitungsschritt nicht verlieren? Beim ersten Import und beim ersten Werkzeug. Jeder gefundene Fehler ergänzt einen Prüffall.	Bei Daten-/Modell-/Werkzeugänderungen Erwartungen fachlich prüfen. Einen fehlschlagenden Test nicht nur passend umschreiben; Ursache klären. Vor dem Release vollständigen Prüflauf ausführen.